

OCR NEA

COMPUTER SCIENCE

PROJECT

NAME:	Vinicius Ferrari
CANDIDATE NUMBER:	
SCHOOL:	Ashby School
TITLE:	Richouse

CONTENTS

ANALYSIS	3
An outline of the problem	3
Stakeholders	3
How the computational methods can solve the problem.....	4
Thinking abstractly	4
Thinking ahead.....	4
thinking procedurally	5
Thinking Logically	7
thinking Concurrently.....	7
Conclusion	7
Research	8
Interviews planning	8
interview script - enzo garofani (translated from portuguese)	9
interview script - alex Michalski (english taken).....	10
interview script - Zach Ward (english taken).....	12
investigating the “bob the robber 2”	14
conclusion	19
Features of the solution	21
Hardware and software requirements	24
hardware requirements	24
software requirements	24
Success criteria	25

ANALYSIS

AN OUTLINE OF THE PROBLEM

For my project I will develop a game inspired by “Bob The Robber 2”, this is a 2D game created by two small developers, Flazm and Meow Beast, and distributed by websites, such as Friv. that the main objective is steal something from a house or another build protected by guarders, cameras and lasers. My game will be a top down with objective and features similar to the “Bob The Robber 2”, but implementing new puzzle features to improve the immersion of the game.

It will demand:

- A map divided by solid walls, for the rooms
- 2 bars in top, one with player’s life and another with money got by player
- Moving sprites, for the guards and/or dogs
- Rotating sprites, for the cameras
- Game over screen
- Menu screen and button

STAKEHOLDERS

The game won’t have any restriction for ages, all of elements will be “family friendly”. It won’t also have limitation for audio, given that it won’t have dialogues, and the music won’t interfere in the gameplay. It will use few buttons for actions, such as W, A, S, D, F and G; and it will be easily adaptable for touch screens because the few buttons needed, but the main objective is the computer players. As it’s inspired by “Bob The Robber 2”, the main public will be persons aged between 14- 18 that probably played the “Bob The Robber 2” in the web browsers.

Given that, my stakeholders are:

- Enzo Garofani, a 17 years-old student from Brazil, I meet him when I lived there, and I played “Bob The Robber 2” with him.
- Alex Michalski, a 17 years-old student from United Kingdom, he’s in my current school and he like this style of game.
- Zach Ward, a 17 years-old student from United Kingdom, he’s in my current school and he played some games in this style.

HOW THE COMPUTATIONAL METHODS CAN SOLVE THE PROBLEM

THINKING ABSTRACTLY

I need to identify and decide what will be necessary for the game, for this I should think in an abstract way, describing the game in a simple way. The abstraction divides the “Richouse” into 4 essential parts:

- Menu

The initial screen of the game, contain 4 buttons: the button to start the mission, after chose one map; the button to open the configurations; the button to open the ranking and the button to quit the game.

- The configuration

User can change basic configurations, such as audio volume and the controls.

- Ranking

Two tables showing the best performances in each of the two maps, the top 10.

- Gameplay

The game start in the front door of the house. The house will be saw in the top, so users can't see the roof, they see the walls dividing the square rooms. The house has security systems such as cameras, guards, lasers and locks. The status bar will be a single bar with time taken and player's life. If guards and cameras bar is yellow, they are identifying you, if red, they already saw you and you lost a life.

THINKING AHEAD

I should be prepared for possible complications and choose the tools to be used. The first step is complete the gameplay, after this I can complete the things with lower priority, the menu screen followed by ranking and finally the configurations. Tools to complete these 4 parts:

- Python with Pygame

I choose it because the python is a widely used language, it allows the use of object-oriented programming to abstract the code and its “sqlite3” library can connect the application with SQLite databases. The “Pygame” is the library that will allow the creation of the GUI in addition to support sprites, collision and

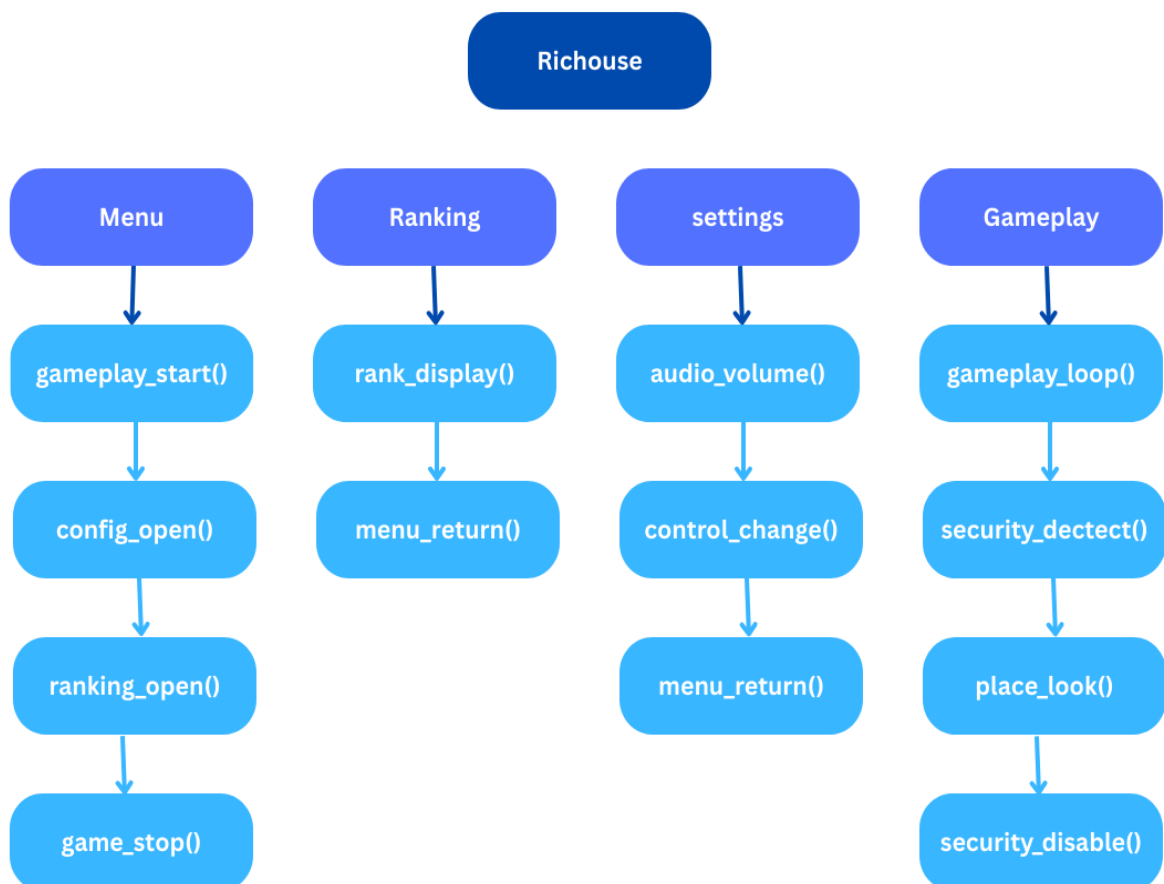
sounds. Python with these libraries will be able to handle and response user's input. With this library I can handle inputs with "pygame.event.get()", detect the collisions with "pygame.sprite.collide_rect()", add music using "pygame.mixer" and add other features without the need of an external tool. The use of external tools, such as engines, can difficult the development, so the ideal is use the Pygame because it's a library of Python easily installable.

➤ SQLite

It allows the creation of multiple tables into a single file, it's better than use plain files. Furthermore, it allows a direct contact with python and its operations allow an efficient handle of database data. Better than a flat file, such as JSON or CSV, because the data is more accessible using simple commands, such as "SELECT ... FROM ... WHERE ..." to get, "UPDATE ... SET ..." to update and "INSERT INTO ... VALUES ..." to add data. Furthermore, these commands are easily executable with sqlite3 module in Python.

THINKING PROCEDURALLY

Look at the structure of game, break it into blocks to clarify the steps and how all of it will work together.



➤ **Richouse**

The name of the 2D top-down game being developed. It will be stealth, user should try to escape from the house with the golden statue, it will define the success of the mission.

➤ **Menu**

Initial screen of the game that use functions: `gameplay_start()` to control the start button starting the mission in the map that user chose; `config_open()` to control the button that open the settings screen; `ranking_open()` to control the button that open the ranking screen; and the `game_stop()` to control the quit button. The functions work when the ENTER is pressed in the button wanted.

➤ **Ranking**

The objective is displaying the ranks so it will use around 2 functions. The `rank_display()` is responsible to display the rank, and the `menu_return()` will be used again here.

➤ **Settings**

It has the function to control the change in the game settings. The `audio_volume()` is responsible for controlling the volume bar, pressing A to decrease and D to increase; the `control_change()` allows the change of gameplay control by pressing the button that you would like to use and pressing ENTER to confirm. Finally, the function `menu_return()` return the user to the menu when the button is pressed

➤ **Gameplay**

The gameplay features will be controlled by a series of functions. The `gameplay_loop()` control the movement, collisions, and all other things that should be updated constantly, responsible by the keyboard inputs, Pygame's main loop; `security_detect()` will control the sprites perception; `place_look()` will control the search feature; and `security_disabled`.

THINKING LOGICALLY

There are some aspects to be considered in the execution to avoid problems.

- The same keyboard buttons should be handled differently when used to move or select button. The condition “if button == ‘W’ and screen == ‘gameplay’” will move forward, already the “if button == ‘W’ and screen == ‘menu’” will select the button on top. It will be handled by `gameplay_loop()`.
- The area where sprites will identify you, guards, dogs and cameras can’t see you if they aren’t looking to you, it can be solved with a variable “wide == true”, the condition “if security_detect() and wide != false” will make the security detect you, but it won’t detect if “wide == true”.
- Make sure that any of the solid parts of map could trap the character or the other sprites. An example is when enter in a room with two doors with “status == ‘locked’”

THINKING CONCURRENTLY

I should consider that some features of the game work at the same time of another ones. Some of them are constantly executed during the game.

- The timer will work at same time of the other features, it will be controlled by `gameplay_loop()`, Pygame’s main loop, using the feature “`pygame.time.Clock`” that allow the concurrency of features.
- Other feature controlled by the `gameplay_loop()` will be the inputs, using the feature “`pygame.key.get_pressed`” to detect the input and handle it considering the variables, such as the “screen” that show which screen is open. It happen while other features are working.
- The music and the audio effects are reproduced by `gameplay_loop()` constantly during the duration of the game. It’s possible because of the feature “`pygame.mixer`” that execute it.

CONCLUSION

Computational methods give the basis of thinking for the project. In the abstraction we can break the problem in steps to be solved, with this I decided to finish the gameplay first, because in this way the stakeholders will be able to test the game features, while it happen I can finish other screens and add design increasing the productivity. Already the thinking ahead I could decide the tools, I chose the Pygame because it has tools to solve both graphical and audio outputs, e.g. using “`pygame.mixer`” for audio and “`pygame.sprite`” for sprites, furthermore it’s an easily installable Python library, so no

external tools needed making the development easier. I also chose the SQLite3 for database because different from flat files, such as JSON, the data can be divided in several tables, such as table for results and other for user data, and it's easily accessible with commands such as "SELECT ... FROM ... WHERE ...". With this information I could divided the game in blocks of functions, it facilitates to think how to implement in Python, each screen has its own specific, rank_display(), or general functions, gameplay_loop(). The gameplay_loop() will control the concurrent features using the "pygame.time.Clock" that allow this concurrency, it will handle considering variables created to avoid problems, such as the "screen" variable.

RESEARCH

INTERVIEWS PLANNING

For the success of the game, it's important to ask for the stakeholders some questions about the ideas given until now. I should know their opinion to know what features are important, besides it I need to know the suitability and acceptance of my planning. I divided into three categories and take some questions for each one.

Question layout:

Gameplay

- Do you like stealth games? If yes, what features make you like it? If no, what make the game boring in your opinion?
- In general games, which buttons do you like to use for the movement? And for the interactions?
- In a robber stealth game, what status should be included? Example: player's lives, or time taken.
- Do you like the audiovisual effects in the stealth games? What is their importance?
- What configurations usually do you change when you play a game?

Ranking System

- What do you think about the games that have a ranking system? How it impacts on the gameplay?
- Which of the status that you wrote above would be ranked?

Design

- Do you like to play 2D top-down games or you prefer other types? What kind of camera position do you like?
- What art styles do you like for stealth games? What is your favorite art style?
- What should be included in a menu screen?

Extra (Optional)

- Are there some other ideas that you want to tell?

INTERVIEW SCRIPT - ENZO GAROFANI (TRANSLATED FROM PORTUGUESE)

Gameplay

- Do you like stealth games? If yes, what features make you like it? If no, what make the game boring in your opinion?

Yes, the suspense creates is good and explore good mechanics in games.

- In general games, which buttons do you like to use for the movement? And for the interactions?

W A S D, E F C.

- In a robber stealth game, what status should be included? Example: player's lives, or time taken.

Player's life, something like visibility, how much you were detected.

- Do you like the audiovisual effects in the stealth games? What is their importance?

Yes, the immersion is very important.

- What configurations usually do you change when you play a game?

None, I play the game in the way that it was programed to be played.

Ranking System

- What do you think about the games that have a ranking system? How it impacts on the gameplay?

I think it's useless depending on game style, unless it's a game against the time or stimulate competitiveness.

- Which of the status that you wrote above would be ranked?

Any of them, I don't see necessity to implement ranking in this game

Design

- Do you like to play 2D top-down games or you prefer other types? What kind of camera position do you like?

They are too good, 2D horizontal games can be good for furtivity games.

- What art styles do you like for stealth games? What is your favorite art style?

Could be more industrial, or more cartoon-styled, pixeled graphics could be strange for this kind of game.

- What should be included in a menu screen?

Something more interactive and dynamic, don't have a static menu values the game, as well as a beautiful art.

Extra (Optional)

- Are there some other ideas that you want to tell?

All games that look for a sensation of identity in their proposal always become an interesting indie to play. Thus, for example, I don't believe that beautiful arts make sense in this kind of game, however a variety of mechanics is always welcome.

INTERVIEW SCRIPT - ALEX MICHALSKI (ENGLISH TAKEN)

Gameplay

- Do you like stealth games? If yes, what features make you like it? If not, what makes the game boring in your opinion?

Yes, I think stealth-based games are very interesting to play. Some of the best features is the strategic side and "ghosting" (going unnoticed)

- In general games, which buttons do you like to use for the movement? And for the interactions?

In my opinion, I like to use WASD for movement and F or E for interactions.

- In a robber stealth game, what status should be included? Example: player's lives, or time taken.

I think you should include player life, amount stolen/taken, time left and how close you are to being detected: for instance if you get seen by an NPC a bar turns yellow and the closer you are it turns to orange and finally red once spotted.

- Do you like the audio-visual effects in stealth games? What is their importance?

Yes, because it gives you one more sense which makes the game feel more real and helps players immerse themselves into the game.

- What configurations usually do you change when you play a game?

Normal, (depending on what game) I would change the button to change tool or weapon to mouse button 4 and 5.

Ranking system

- What do you think about games that have a ranking system? How does it impact on the gameplay?

I think games can't go wrong with a ranking system as it adds an additional achievement to reach. It makes the player more competitive.

- Which of the statuses that you wrote above would be ranked?

Amount stolen: greediness rank

Time left: swiftness rank

Detection: stealthiness

Design

- Do you like to play 2D top-down games or do you prefer other types? What kind of camera position do you like?

My personal favorite is first person, but I do enjoy a good 2D top-down game as they feel retro. I like one where the camera is close and follows the character as if you could see the whole map and whereabouts of enemies it could make the game too easy and less enjoyable.

- What art styles do you like for stealth games? What is your favorite art style?

For a stealth game I think you could use most styles. But it depends if you want your game to be scary go for dystopian realistic style, but you want a relaxing casual game go for a cartoon style. My favorite is realistic.

- What should be included in the menu screen?

Start button, quit button, options button: where you can reconfigure key binds and change volume of sound effects or music, you could also add a credits button.

Extra (optional)

- Are there some other ideas that you want to tell?

You could add vanity where you can change how your robber looks and make him unique.

You could also have a story line for the robber about his past to make the player feel sympathy, so they enjoy the game more.

INTERVIEW SCRIPT - ZACH WARD (ENGLISH TAKEN)

Gameplay

- Do you like stealth games? If yes, what features make you like it? If not, what makes the game boring in your opinion?

I enjoy stealth games that create tension and constant fear of being caught, the game can become boring without this added fear. There must be a variety of ways to attempt each objective to create a sense of freedom that comes with the stealth genre

- In general games, which buttons do you like to use for the movement? And for the interactions?

WASD, E for interact, I for inventory, numbers 1 - 9 for equipment or items

- In a robber stealth game, what status should be included? Example: player's lives, or time taken.

Time Taken should be included at the end of the mission, maybe items stolen, value of items, most valuable item stolen should be included at the end of the mission, along with a letter rank or a score from 0 - 100 based on the success of the mission

- Do you like the audiovisual effects in stealth games? What is their importance?

Simple effects, notifies that user which objects and interactions make noise/could alert people nearby. Too complicated HUD icons or sound effects could break the user's immersion into the stealth game.

- What configurations usually do you change when you play a game?

Mouse Sensitivity, Graphics quality e.g. High, Medium, Low. In-game resolution.
Option to change your controls.

Ranking System

- What do you think about games that have a ranking system? How does it impact on the gameplay?

If rewards are offered, it creates a strong incentive to play and a competitive nature in the game, however, the use of bugs and cheats by other users could inflate the ranking system.

- Which of the statuses that you wrote above would be ranked?

Individual mission performance, total performance ranked across every mission.
Potential for an online mode where users compete to rob the same area and can attempt to stop each other through e.g. weapons and gadgets

Design

- Do you like to play 2D top-down games or do you prefer other types? What kind of camera position do you like?

For stealth games, I've usually only seen 3D first person or third person, so a 2D top-down stealth game would be really innovative and potentially easier to code.

Thinking of something similar to Hotline

Miami: <https://youtu.be/Kqr0yUuSiTs> Hotline Miami is much more fast paced, but the camera and art style would be unique for a stealth game.

- What art styles do you like for stealth games?

I've only seen more realistic, 3D stealth games made before, see Thief Simulators and Hitman:

https://store.steampowered.com/app/704850/Thief_Simulator/

https://store.steampowered.com/app/1659040/HITMAN_World_of_Assassination/

- What is your favourite art style?

I am a fan of the Hotline Miami style; it would be easier to create art for and code while being unique and differentiated itself from other stealth games.

- What should be included in the menu screen?

- Start/Play/Missions

- Ranked

- Options
- Achievements
- Extras e.g. credits

Extra (Optional)

- Are there some other ideas that you want to tell?

Hope the game development planning goes well, take some good inspiration from Thief Simulators on steam, they have nailed the robber stealth game mechanics e.g. lock picking, safe cracking, detection.

INVESTIGATING THE “BOB THE ROBBER 2”

I played the “Bob The Robber 2” in *Friv*, with this I could remember the mechanics of the game. I will use adaptative versions of these features and I will implement new features, using puzzles. I took some screenshots to analyze the feature and plan some adaptation.

Menu Screen:



A simple menu screen, it just has the “play game” button to start the game. It doesn’t have a specific screen for configurations, it just shown the audio intensity of top right and the option to mute the music. It has an attractive art that shown intuitively the main objective of the game. I chose to use this style with all options and the logo in same column, I will just add the configurations and ranking button.

Player status bar:



It includes the time taken in phase, represented by clock. There is also the money amount stole, represented by the money icon. Furthermore, it uses alarm icons to show how many times the player was got by security system. From this status, I will use the alarm (lives) and the time, I won't include the money because the main objective will be stealing a golden statue as quick as possible. The house furniture will have items to be used in puzzles, instead of money.

Map:



It has a map with different rooms, each one with different widths and same height. It also has the arrows showing where the main objective is. There are different kind of mobilities, lifts and stairs. For my game, the rooms will be all squares or rectangles, but with different sizes, that will fit on map. Furthermore, I prefer to use a red circle to show the objective, like as a house map demarcation. I won't include lift and stairs, because of the game will be top-down.

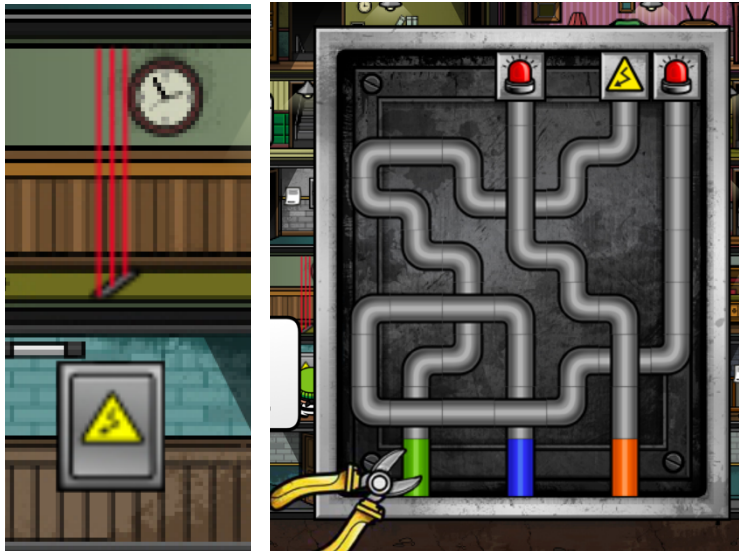
Security System:



The game has locked doors, I used a lockpick to open, I align the middle metal using the left and right arrows and press the up arrow to unlock, it make me use more the agility to press the button in the right moment. In my game, the lockpick could be used on horizontal, pressing “F”, align the lockpick with pines using “A” and “D”, and press “W” to press each pine until all of them be pressed.



It uses doors with passwords, I pressed “W” to open the panel, it make me think about where is the password, in this way the game become more challenging. In my game, it could be a digital panel, and I could implement a system to put some flour in it to reveal the password used.



It uses lasers with a system of lasers that can be disabled using its panel, it uses “W” to open the panel, it make me feel like an eclecticist. In my game, the system of lasers and cameras could be controlled online, so the user could press a specific button to open the character hacker tool to crack the house system, as result the security system is disable during a time.

Lives, game over and mission completed



It counts the lives using alarms, each time that the security system got me, the alarm turns on. When this happen, I lost one live and appear some audiovisual effects connected with alarm, increasing the tension when I was got. I will remain this idea for my game.



When I was got 4 times, a game over screen appears, but instead of a write of “Game Over”, an image showing the destiny of my character appear. It is important to remain the user in the immersion even when they lose.



When I finished a phase, a journal screen appears, like as my character was seeing its own crime on TV, it increases the feel of success on me. Therefore, I will inspire on these screens to make my own game over and mission completed screens without just writing phrases.

CONCLUSION

After the research I should group all ideas from interviews and game research. It supports in the decision of features.

Feature idea	From
Gameplay	
Stealth game	Developer, all stakeholders agree
Security systems	
Guards, dogs or robots	Game research
Locks	Game research
Cameras	Game research
Lasers	Game research
Door with code	Game research
Buttons	
Movement: W, A, S and D	All stakeholders; Game research
Interactions: E and F	All stakeholders; Game research
Statuses	
Player's life	Alex and Enzo; Game research
Stealthiness	Enzo and Alex
Amount stolen	Alex and Zach; Game research
Time taken	Zach; Game research
Most valuable items	Zach
Audiovisual effects	
Notify what kind of object was got	Zach
Interactions have effects and affect sprites	Zach
Ghosting, hide in places	Alex
NPC bar turn yellow when start to see you	Alex; Game research.
Configurations	
Option to change graphics	Zach
Option to change controls	Zach
Option to change sound effects	Alex

Ranking	
Should be included	Alex and Zach
Status to be ranked	
Amount stolen	Alex
Time taken	Alex
Stealthiness	Alex
Design	
Symbols representing statuses	Game research
Camera	
Top-down	Enzo, Alex and Zach
Horizontal	Enzo
Art style	
Realistic	Alex and Zach
Cartoon	Enzo and Alex
Pixel	Zach
Screens	
Game over/mission complete	
Unique art	Game research
Menu button	Game research
Replay button	Game research
Menu screen	
Dynamic	Enzo
Start button	Alex, Game Research
Quit button	Alex
Options button	Alex
Credits button	Alex; Game research
Change your character	Alex
Attractive art	Enzo; Game research
Map	
Rooms with similar size	Game research
Difference mobilities	Game research

FEATURES OF THE SOLUTION

After I see all suggestion from the stakeholders interview, I should select the most interesting features ideas and find a way to implement it.

Feature	Justification
Screens	
Menu	A screen before the game start, it can't just start.
Start button	Start to play one of the 2 maps, chose by player
Rank button	Access the ranking with the best scores of the game
Options button	Where user press to access the configuration of the game
Quit button	It will stop the program when pressed.
Dynamic	More interactive menu makes the game more interesting for the user. Each button is selected using W for up and S for down, press ENTER to select
Select character	User will be able to select the pre-designed sprite that they most like, press A to change to the left character and D for to the right
Configuration	The user would be able to personalize some game features to their liking
Option to change volume	Each user has a different preference for the volume of audio, increase using D and decrease using A
Option to change controls	Each user has a different preference for the buttons used, it includes the selection buttons listed above.
Ranking	A table showing the best scores of the game, all store in a SQLite database. It increases the game competitiveness. 2 ranks, one for each map

First criteria: Player' life	How many times the player was found, if they weren't found, the life will stay the same. Most important of a stealth game.
Second criteria: Time taken	Fewer is the time taken per mission, better is the thief.
Game over/mission complete	A way to show to the user that mission was finished. Use A and D to select the button to be pressed.
Continue button	It will redirect the user to the menu screen.
Restart button	It will restart the mission in the same map.
Gameplay	
Control buttons	The game should have few buttons as possible and should be common, with this the user gameplay become easier.
Movement: W, A, S and D	Most common buttons for the movements, W to go forward, S to go backward, A to go left and D. to go right. Users often already know it.
Interactions: E and F	The buttons E and F are close to the movement buttons, it makes easier to press.
Stealth	The game will have the focus on stealth, the player shouldn't be found by security systems. It creates a tension resulting in a higher immersion for the user.
Security Systems	They have the objective to search and found any criminal. If they found the user, one live is lost.
Guards and dogs	The house will have sprites locomoting and searching for any criminal. Inside will have guards and outside will have dogs.
Locks	It will lock some doors. The users will be able to open using a lockpick by pressing the pines inside it.
Cameras	It will be a fixed and rotating sprite. It will relate to house system; user will be able to hack the system to disable it.

Lasers	It will block some places, including the main vault. It will also be linked with house system.
Doors with code	The door will have a digital panel. User will be able to use a UV light to see the fingerprint, they will press in crescent order of intensity.
Statuses bar	A bar showing information about the user during the mission.
Player's life	The number of times that user can be found until lose. Represented by an alarm symbol.
Time taken	The time taken in the mission. Represented by a alarm symbol.
Stealthiness perception	The guards and cameras will have an indicator up their heads, it will change the color when the player is precepted or found.
Places to hide	The player will be able to enter inside some places to hide. Guards, dogs and cameras won't be able to see.
Audiovisual effects	
Audio	The game will have a music and an alarm sound, when player is found, to increase the immersion.
Visual	The game will have effects when perform an interaction and when the player is found.
Design	
Symbol for statuses	The game will have symbols to represent the statuses in the bar. The time will be a clock and the player's life will be an alarm.
Top-down camera	The camera will be top-down because it's an uncommon camera style for stealth game, an innovation for the it.
Cartoon art style	A common art style for the 2D games and it will be easier to implement.
Map with square rooms	The map will be constituted by square rooms of different sizes, It will support on the implementation of features.

HARDWARE AND SOFTWARE REQUIREMENTS

HARDWARE REQUIREMENTS

Hardware	Reason
Keyboard	For buttons to move, interact with map and to access the menu.
Monitor	To display the graphics of the game, it includes the special effects. The recommended is 1280x720
Speakers/headphones	To play the music and the audio effects of the game
500MB of hard drive storage	For the Visual Studio Code and the files with the game code
2GB of RAM DDR3	Minimum requirements to install Visual Studio Code
1.6GHz processor	Minimum requirements to install Visual Studio Code

SOFTWARE REQUIREMENTS

Software	Reason
Windows 10/ macOS 10.12/Linux	Minimum requirements for the VS Code
Visual Studio Code	The coding editor used

Python 3.6 or higher	The computer language used, I used 3.6 version, but can be higher. It's installed with VS Code
Pygame 2.0 or higher	The Python library used to develop the graphics and sound of the game, the version used was the 2.0, can be higher. It's installed on python with command pip.

SUCCESS CRITERIA

Requirement	Justification	Reference	Completion status
Menu screen	The game needs a screen before starting the game, it will give access to other features	Game research	Not completed
Button selection with W and S	It will enable the user to select one of buttons in the column. W is up, and S is down. The game doesn't use mouse because it turns easier to adapt to consoles in the future.	Proposed solution	Not completed
The option button opens the configurations screen	It will enable user to open the configuration screen from menu.	Alex's interview	Not completed
The ranking button opens the ranking	It will enable the user to open the player's rank from the menu.	Proposed solution	Not completed
The start game button starts the mission	It will enable the user to choose one of the two maps and start it.	Game research and Alex's interviews	Not completed
The quit button stops the program	It enables the user to leave the game.	Alex's interview	Not completed

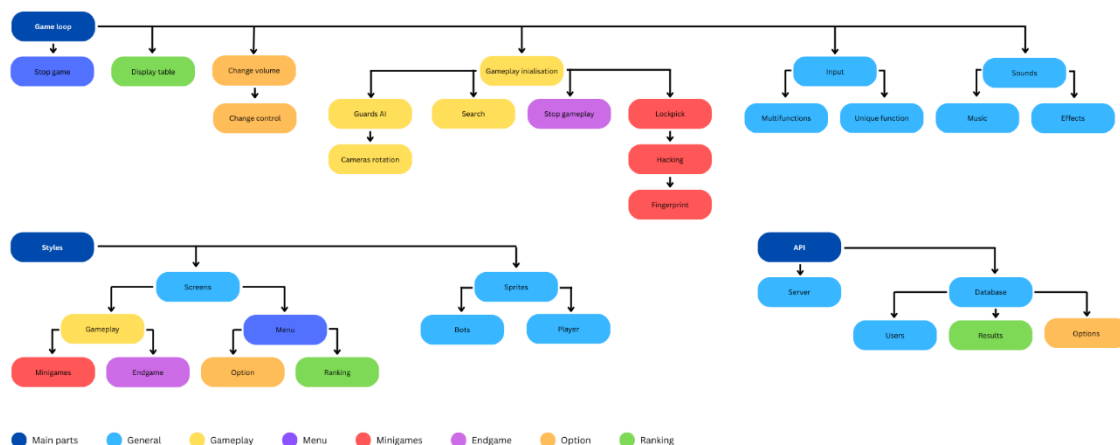
Ranking screen displaying the rankings.	It will display two rankings with the best times reached by players in each map. It creates a competitiveness.	Alex's interview; Zach's interview.	Not completed
First criteria to be ranked is the player's life, after is the time taken.	The most important thing in a stealth game is complete the mission without be precepted and quicker as possible.	Alex's interview.	Not completed
Exit button leaves from ranking screen.	It enables the user to return to the menu screen.	Proposed solution	Not completed
Configurations screen	It enables the user to select and change the configurations.	Proposed solution	Not completed
Configuration selection with W, S and ENTER	It enables the user to select the configuration in the column. W go up, S do down and ENTER select. The game doesn't use mouse because it turns easier to adapt to consoles in the future.	Proposed solution	Not completed
Bar with audio volume	It will enable to change the audio by changing a bar. Use A to decrease, D to increase and ENTER to confirm. The game doesn't use mouse because it turns easier to adapt to consoles in the future.	Alex's interview	Not completed
Change control option	It enables the selection of different buttons from	Zach's interview	Not completed

	keyboard for the gameplay.		
Exit button leaves the configuration screen.	It enables the user to return to the menu screen.	Proposed solution	Not completed
Keyboard buttons for all action in the game	It makes easier to adapt the game for consoles in the future.	Proposed solution	Not completed
Press W moves forward, S moves backward, A moves left, and D moves right.	It's the standard button for movement in many games. It can be personalized at configurations	All interviews; Game research.	Not completed
Press F searches the places for items, and E uses the item.	Makes easier to press because they are close to the movement buttons.	All interviews; Game research.	Not completed
Items inside places to be searched.	Drawers and cabinets will hide the items necessary to disable security systems, they can be empty.	Proposed solution	Not completed
Security systems looking for the player or blocking places.	Their function is looking for the player or block rooms, it turns the mission harder.	Game research.	Not completed
Guards and dogs moving across the map.	It makes the user move across the map instead to stay in same place. Make the game more dynamic.	Game research.	Not completed
Guards and dogs disable temporarily after kicking them from behind.	It gives opportunity for users enter in certain places before they return. No items needed.	Game research.	Not completed
Cameras rotating in same place.	Makes the user be more attentive in some areas.	Game research.	Not completed

Lasers blocking some places.	Makes the user find some way to disable it.	Game research.	Not completed
Lasers and cameras temporarily disabled when house system is hacked, with the tablet minigame.	Turns the game dynamic because the user should find an item to hack it with the tablet, need to be found.	Proposed solution.	Not completed
Locks blocking some rooms.	Make the user find a new path to pass.	Game research.	Not completed
Minigame of the lockpick used to press the pins inside the lock to open it.	Create the possibility to open paths and turn the game harder, because we need to find the lockpick.	Proposed solution.	Not completed
Doors locked with code.	A second variation to block the door and turn the game harder.	Game research.	Not completed
Minigame to find the code by using UV light in the panel, press in order of intensity.	Common feature shown in movies. Turn harder because the user should find the UV light.	Proposed solution.	Not completed
Player statues bar at top.	Shown the user statuses and record it for the rank.	All interviews; Game research	Not completed
Shown the player's life.	Important to know how many tries the player has.	Enzo's interview; Alex's interview; Game research.	Not completed
Shown the time taken.	Important to create competitiveness.	Zach's interview.	Not completed
Lost one life when security system identifies the player.	Create a limit of tries.	Proposed solution.	Not completed
Guards, dogs and cameras change the bar color up them when start to see the player.	Create an alert for the user know that they will be identified if stay there.	Alex's interview; Game research.	Not completed
A golden statue is the main objective.	There is no money bar because it will		Not completed

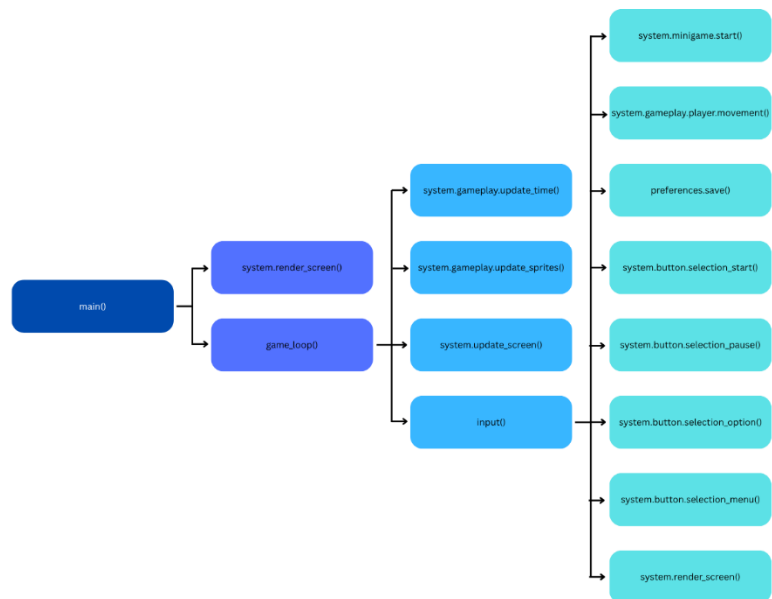
	define the success of the mission.		
Game over screen appear when user lost all lives.	Give the use a feel of fail.	Game research.	Not completed
Mission completed when the user leaves the house with the main objective.	Give the user a feel of success.	Game research.	Not completed
Cartoon graphics	Common graphics style for 2D games. Easier to develop than realistic or pixeled.	Alex's interview; Enzo's interview.	Not completed

DESIGN



I use the diagram to divide the design in three main parts. The first part is the game loop, here is where the actual game code work, it includes anything that is executed in the user computer. The second part is the styles; all the style of screens and sprites will be design here. Finally, the last main part is the API, it's all code that don't run in user computer, it because we should have the access to some information in real time and it's dangerous to give the users information in the distribution code. The other parts are separated in colors; it supports the link of parts between the main sections.

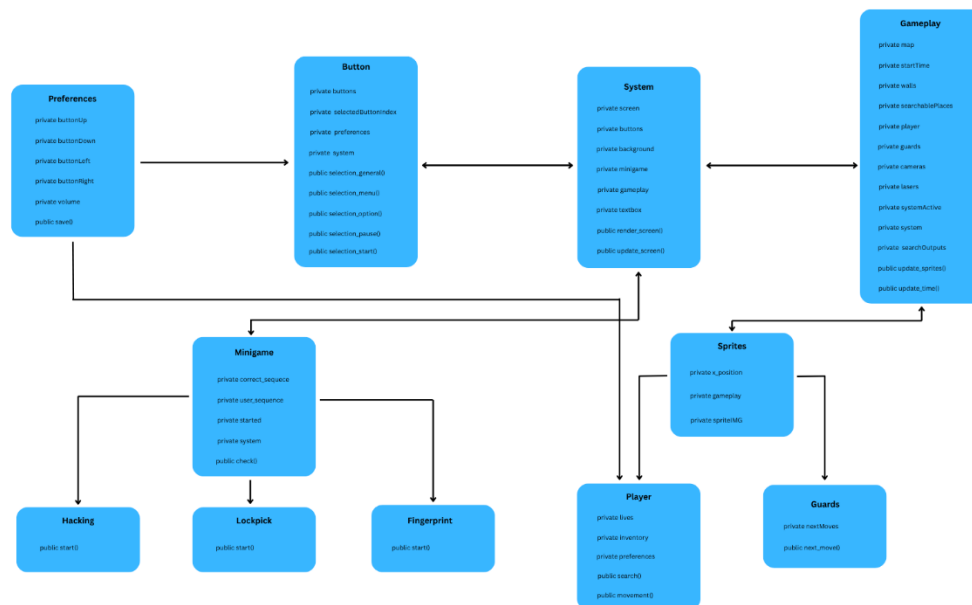
DISTRIBUTION CODE



The distribution code is the game code that run on user's machine. I chose to divide the explanation in parts of pseudocode; it makes easier for the development part given that the computational logic is already applied. The diagram below shows how the functions, procedures and methods link together. It shows each layer of the code; it doesn't include any function inside the methods used. Each layer has a color.

CLASSES

I divide the code in multiple classes, it will support me to divide the features code making easy to implement, reuse and test the code. The diagram below show the private attributes and public methods of each one. The arrow show the relationships that could be inheritance, I use because it is useful when no existent class attributes are used, or composition, I use because it is useful when the attribute to be used is from an existent class.



Inheritance:

- Minigame -> Lockpick, Hacking and Fingerprint
- Sprites -> Player and Guards

Composition:

- Gameplay, Button and Minigame -> System
- Sprites -> Gameplay

Aggregation:

- Preferences -> Button and Player
- System -> Gameplay, Minigame and Button
- Gameplay -> Player

VARIABLES AND DATA STRUCTURES

It's important to understand the variables and data structures used to understand the work of the pseudocode. They can be present inside a class, so they are attributes.

Name	type	Scope	Explanation
isrunning	Boolean	Global	It defines if the main loop should continue. With it we can start and stop the game securely.

system	Object	Local	Object with attributes used to store states of the game and create relationships.
preferences	Object	Local	Store the object with the user preferences got in API
System class			
screen	String	Class	Store the current screen state
background	Object	Class	Store the background to be rendered. It's a file.
button	Object	Class	Create a composition relationship between System and Button
gameplay	Object	Class	Create a composition relationship between System and Button
minigame	Object	Class	Create a composition relationship between System and Minigame
textbox	String	Class	Store the text input
Preferences class			
buttonUp	String	Class	Store the preferred button to select up or move upwards
buttonDown	String	Class	Store the preferred button to select down or move downwards
buttonLeft	String	Class	Store the preferred button to move left

buttonRight	String	Class	Store the preferred button to move right
volume	Integer	Class	Store the preferred volume of sound
Button class			
buttons	List	Class	A list of names that will be print as buttons, the order of buttons is essential
selectedButtonIndex	Integer	Class	The index of the button selected, it show the position of it in buttons
preferences	Object	Class	Create a aggregation relationship between Preferences and Button
system	Object	Class	Create a aggregation relationship between System and Button
Gameplay			
map	String	Class	Store the current map being played
startTime	String	Class	Store the start time of the mission
walls	List of Tuples	Class	Store the coordinates of all the walls in map. Tuple store the coordinate.
SearchablePlaces	List of Tuples	Class	Store the coordinates of all searchable places. Tuple store the coordinate

player	Object	Class	Create a player sprite with the composition relationship between Player and Gameplay
searchOutputs	List	Class	The list of possible outputs when player search
guards	List	Class	List of objects, the guards' sprites. Use the composition relationship between Minigame and Gameplay, relationship passed across by inheritance.
cameras	List	Class	List of objects, the cameras' sprites. Use the composition relationship between Minigame and Gameplay, relationship passed across by inheritance.
lasers	List	Class	List of objects, the lasers' sprites. Use the composition relationship between Minigame and Gameplay, relationship passed across by inheritance.
systemActive	Boolean	Class	Store the current state of the alarm
system	Object	Class	Create an aggregation relationship

			between Gameplay and System
Sprites			
coordinate	Tuple	Class	Store the current sprite coordinate
gameplay	Object	Class	Create an aggregation relationship between Gameplay and Sprites
spriteIMG	Object	Class	Stores the file with the image of the sprite
Guards			
nextMoves	Queue	Class	It stores the next 5 moves for the guards
Player			
Lives	Integer	Class	Store the number of remaining lives
Inventory	List	Class	Store the items got from searchable places
preferences	Object	Class	Create an aggregation relationship between Player and Preferences
Minigames			
correct_sequence	List	Class	Store the correct sequence of the minigame
user_sequence	List	Class	Store the user input sequence
started	Boolean	Class	Store the minigame state
system	Object	Class	Create an aggregation relationship with System

I chose to use most of the variable to be attributes because it allows multiple function to use them, we just pass the object as parameter turning the code easier. The “isrunning” is global because is the main control of the game state, so we can stop the game anywhere. The other local variables are used to create the objects. I use list in the big part of data structures because it’s a build-in Python tool, so the development is easier. The only data structure that isn’t a list is the queue, but it will be implemented using lists.

MAIN()

It’s the initialization of the code. It initializes the tool, create the screen and the title, and start the background music that will stays the same for the whole game. Here the first interaction with database happens, it’s a local database that store the user preferences. The creation of system variable is essential to control the screen states. Preferences will store the user preferences about buttons and send it to API.

```
procedure main()

    initialize pygame

    create screen 600 x 600

    create title "Richouse"

    play music forever "music.mp3"

    select user data from database

    if user data is null then

        system = System()

        system.render_screen("account")

        preferences = Preferences(standard preferences)

    else

        preferences = Preferences(data from database)

        system = System()

        system.render_screen("menu")

    endif

    game_loop(system, preferences)

endprocedure
```

I separated the system and preferences objects because the preferences would relate with multiple classes, so it’s better to create an own class to make these relationships easier. The game loop is separated also because in this way I can test different parts of code inside the main() function without need to use the game_loop(). The screen will be 600 x 600 because will be easier to develop the pathfinding algorithm. The audio file will be mp3 because I will use some existent music and it’s the most common file type for

it. There is a local database beside the API one because it doesn't depend on server, using less server processing. We can see that using Pygame it can be easily executed with few commands making more reliable to use it.

GAME_LOOP()

This is the main loop of the game, so it controls the things that should be constantly updated. It includes the current screen, the position of the sprites, the time taken during the phase and the inputs entered.

```
procedure game_loop(system: object, preferences: object)

    while isrunning

        system.update_screen()

        if system.screen in ["gameplay", "hacking", "lockpick", "fingerprint"] then

            system.gameplay.update_sprites()

            system.gameplay.update_time()

        endif

        input(system, preferences)

    endwhile

endprocedure
```

I divide the loop in four parts that update things that should be constantly updated such as the screen, the sprites, the time and input handle. Sprites and time taken are only update during the mission.

INPUT()

It will handle the inputs entered by the user. The input is gotten by Pygame, another valuable feature of this tool. I divide the input into screens because the input will be difference for each screen. The classes methods are used because the code become reusable.

```
procedure input(system: object, preferences: object)

    for key in keysPressed

        switch screen:

            case "menu":

                system.buttons.selection_menu(key)

            case "rank":

                if key == "ESC":

                    system.render_screen("menu")

                endif

            case "option":

                system.buttons.selection_option(key)

        endswitch

    endfor

endprocedure
```

```

if key == "ESC" then

    preferences.save()

    system.render_screen("menu")

endif

case "gameplay":

    system.gameplay.player.movement(key)

    if key == "F" then

        interact with object searching it or open a minigame

    else if key = "ESC" then

        system.render_screen("pause")

    endif

case "hacking":

    switch key:

        case "E":

            If "chip" in system.gameplay.player.inventory then

                system.minigame.start()

            endif

        case "ESC":

            system.minigame.started = False

            system.render_screen("gameplay")

        default:

            add key to system.minigame.user_sequence

case "lockpick":

    switch key:

        case "E":

            If "lockpick" in system.gameplay.player.inventory then

                system.minigame.start()

            endif

        case "ESC":

            system.minigame.started = False

            system.render_screen("gameplay")

        default:

            add key to system.minigame.user_sequence

case "fingerprint":

    switch key:

        case "E":

            If "UV light" in system.gameplay.player.inventory then

                system.minigame.start()

```

```

endif

case "ESC":

    system.minigame.started = False

    system.render_screen("gameplay")

default:

    add key to system.minigame.user_sequence

case "pause":

    system.button.selection_pause(key)

case "account":

    if key == "Enter" then

        send username to API

        if username don't exist then

            save in local database

        else

            print("Username already exist")

        endif

    else:

        system.textbox = system.textbox + key

    endif

case "start":

    system.button.selection_start(key)

next key

endprocedure

```

The menu, option, pause and account handle inputs only for selection of buttons, all buttons are in vertical because in this way the code of selection can be reused. The gameplay handle inputs related to player sprite movement and interactions; there is a button to open the pause screen. The fingerprint, hacking and lockpick have inputs that are used to complete the minigame by completing sequences. Account stores the key in a textbox that will be the username. Given that multiple function I divide it into methods of classes that link one with another, so one of motifs to use Python is because of it's a Object Oriented programming language. Something to consider is the authentication of user input in username textbox, some attacks such as SQL injection or XSS attack could be a problem.

PLAYER CLASS METHODS

It controls the player sprite actions, it includes the movement and the search actions.

```
public procedure movement(key: string)

    Switch key:

        case "W":

            position[0] = position[0] + 5

        case "S":

            position[0] = position[0] - 5

        case "A":

            position[1] = position[1] - 5

        case "D":

            position[1] = position[1] + 5

endprocedure
```

The first method is the movement() that handle the movement of player based on the input entered. The Pygame lead the screen as a cartesian graph, so it's easy to control using coordinates. I use tuples to represent the coordinates because it easier if consider a 2d cartesian graph.

```
public procedure search()

    get at random an element of gameplay.searchOutputs

    add item to inventory

    remove item from gameplay.searchOutputs

    remove place from gameplay.searchablePlaces

endprocedure
```

The other function that control the player actions is the search() that is executed when we search one of the searchable places, so the function use the relationship with the Gameplay to see the possible outputs of the search.

BUTTON CLASS METHODS

This class has methods related to buttons selection. The selection is based on a list and the selected button is an element of the list with certain index, it makes easier to lead with button logic.

```
public procedure selection_general(key: string)

    switch key:

        case preferences.upButton:

            selectedButtonIndex = (selectedButtonIndex + 1) MOD LENGTH(buttons)

        case preferences.downButton:

            selectedButtonIndex = (selectedButtonIndex - 1) MOD LENGTH(buttons)
```

```
endprocedure
```

The selection can be general, used to select buttons up and down, or exclusive, that extend the general. All buttons are vertical because I can reuse this general code in all exclusive selections.

```
public procedure selection_menu(key: string)

    selection_general(key)

    if key == "ENTER" then

        switch buttons[selectedButtonIndex]:

            case "start":

                system.render_screen("start")

            case "rank":

                system.render_screen("rank")

            case "option":

                system.render_screen("option")

            case "quit":

                isrunning = False

        endif

    endif

endprocedure
```

The next we can see the extension of menu selection, 4 possible buttons that do different actions. The isrunning is used again without be passed as parameter because it's global, in this way user can stop the game.

```
public procedure selection_option(key: string)

    selection_general()

    if SHIFT is pressed then

        switch buttons[selectedButtonIndex]:

            case "forward":

                preferences.upButton = key

            case "left":

                preferences.leftButton = key

            case "down":

                preferences.downButton = key

            case "right":

                preferences.rightButton = key

            case "use":

                preferences.useButton = key

            case "interact":

                preferences.interactButton = key

        endif

    endif

endprocedure
```

```

endif

if buttons[selectedButtonIndex] == "volume" then

    switch key:

        case "A":

            preferences.volume = preferences.volume - 5

        case "D":

            preferences.volume = preferences.volume + 5

    endif

endif

endprocedure

```

The selection in option is used to change volume and change the preferred controls. When user press SHIFT + key in one of the buttons controlling the action button preferences, this pressed button will be placed as preferred. The volume button will change the volume settings by increasing the volume by 5 when pressed right and decrease 5 when pressed left. The preferences is not a parameter because the attribute used to make the aggregation relationship.

```

public procedure selection_pause(key: string)

    selection_general(key)

    If key == "ENTER" then

        switch buttons[selectedButtonIndex]:

            case "replay":

                system.render_screen("gameplay")

            case "menu":

                system.render_screen("menu")

        endif

    endif

endprocedure

```

Selection of buttons when you pause the screen, it allows the user return to menu or just stop the gameplay temporally.

```

public procedure selection_start(key: string, system)

    selection_general(key)

    if key == "ENTER" then

        switch buttons[selectedButtonIndex]:

            case "map1":

                start the map 1

            case "map2":

                start the map 2

        else if key == "ESC":

            system.render_screen("menu")

        endif

    endif

```

endprocedure

Control the selection of maps in menu, it makes easier to extend the number of maps in future. We see that ENTER was used as the confirmation of selection, it's the standard button of selection in many software, so it's intuitive. For the same motif, the ESC was chose as the return button. W, A, S and D are used both for selection and movement to minimize the number of buttons and make it intuitive, easy for a possible update to consoles in future.

SYSTEM CLASS METHODS

This class control screen state, so its methods are related to screen updates and rendering. Screen should be constantly update because users input change its state. I will use PNG files instead of JPEG because it uses lossless instead of lossy, so the graphic quality will be improved and won't be a problem given that images won't be transferred over internet; another motif is because I'm using Canva because of its simplicity and it's a free tool to make graphic elements.

```
public procedure render_screen(nextScreen: string, userPreferences: object)

    switch nextScreen:

        case "menu":

            buttons = Button(buttons=["start", "rank", "option",
            "quit"],preferences=userPreferences, system=self)

            background = menu.png

        case "rank":

            buttons = NULL

            access API

            background = rank.png

        case "option":

            buttons = Button(buttons=["volume", "forward", "backward", "left", "right", "use",
            "interation"], preferences=userPreferences, system=self)

            background = option.png

        case "gameplay":

            if screen == "start" then

                map = buttons[selectedButtonIndex]

                buttons = Null

                if map = "map1" then

                    gameplay = Gameplay(map=map, startTime= current time , walls= walls
                    of map 1, searchablePlaces= serachable places in map 1)

                    player = Player(add initial place for map 1)

                    create guards sprites with Guards(add initial place for map 1)

                    create lasers and cameras using Sprites(add places for map 1)

                    background = map1.png
```

```

else

    gameplay = Gameplay(map=map, startTime= current time , walls= walls
of map 2, searchablePlaces= serachable places in map 2)

    player = Player(add initial place for map 2)

    create guards sprites with Guards(add initial place for map 2)

    create lasers and cameras using Sprites(add places for map 2)

    background = map2.png

endif

add sprites to gameplay.sprites

start the threading of method next_move() in guards

endif

case "fingerprint":

    buttons = Null

    minigame = Fingerprint(random sequence in range 0 to 9)

    background = fingerprint.png

case "lockpick":

    buttons = Null

    minigame = Lockpick(random sequence of arrows)

    background = lockpick.png

case "hacking":

    buttons = Null

    minigame = Hacking(random sequence of letters)

    background = hacking.png

case "pause":

    buttons = Button(buttons=["replay", "menu"], preferences=userPreferences, screen=self)

    background = pause.png

case "start":

    buttons = Button(buttons=["map1", "map2"], preferences=userPreferences, screen=self)

    background = start.png

case "account":

    background = account.png

    textbox = ""

screen = nextScreen

print background

print rank scores if in rank screen

print sprites and status bar if in gameplay screen

print textbox if in account screen

print the buttons, different color for the selected

```

```
endprocedure
```

Control the initialization or resume of the screens. The menu, option, account, start, pause, hacking, fingerprint and lockpick will always initialize the same. Already the rank will depend on API results to make the rank, results should be in API because we should know the highest score including all player in the world, because it the username is unique. The gameplay will depend on the map choice or if the game was started or just resumed, if resumed, the sprites and time will continue in the same state as before the start screen. I will use the request library to access the API because it's a build-in Python tool, makes the development easier.

```
public procedure update_screen()

    print background

    print the buttons, different color for the selected

    print textbox if in account screen

    print the status bar if in gameplay or any minigame screen

endprocedure
```

I just update the current states using the same thing of `render_screen()` but changing the selection because the button selected constantly change. It doesn't lead with sprites and time printing, other functions are responsible for it. Using background as attribute allow us to update without use parameters. The button selected is know by System because the composition relationship with Button.

PREFERENCES CLASS METHOD

It has only one method that is responsible to update the local database with the preferred controls.

```
public procedure save()

    update the database preferences with attributes

endprocedure
```

The local database store only the preferences and the username. To access it the sqlite3 library will be the best option because of its simplicity and it's build-in Python tool. SQLITE is better for it because it uses few space and work well for the simple specs of this application.

GAMEPLAY CLASS METHODS

It's responsible to update the elements that only happen while the gameplay in running.

```
public procedure update_sprites()

    print the position player

    print the next position of guards

    print the cameras position but rotate
```

```

        if systemActive == true then

            print lasers positions

        endif

        if player touch laser or player within 50px in front of guards or active cameras front then

            sprites return to initial position

            player lost a live

            alarm sound effect happen

            if player.lives == 0 then

                print game over screen

                wait few seconds

                system.render_screen("menu")

            endif

        endif

        if player is in door and have the golden statue then

            print mission completed screen

            send results to API, if it's the higher

            wait few seconds

            system.render_screen("menu")

        endif

    endif

endprocedure

```

The player, guards and cameras are always print because they can disappear, already the laser only appear if system is active. We should check if the player position can be seen by security system, if yes the player lost a live. If player lost all lives then the game will finish with game over and no results are stored. If the player arrive at front door with the golden statue, the mission completed screen appear and the results are stored case they are the higher one. I will use the Python time library to wait these few time.

```

public procedure update_time()

    timeTaken = current time - startTime

    print timeTaken

endprocedure

```

It will update and print the current time taken in the mission, can be calculated using the Python datetime library. The time is only printed during the gameplay or minigames.

MINIGAME CLASS METHOD

It contains only one method that will be inherited by Fingerprint, Hacking and Lockpick.

```

public procedure check()

    length = LENGTH(user_sequence)

```

```

        for x=0 to length
            if user_sequence[x] != correct_sequence[x] then
                wrong sound effect is executed

                system.render_screen("gameplay")

            else if length == 4 then
                success sound effect is executed

                system.render_screen("gameplay")

            endif
        next x
    endprocedure

```

It uses the list of inputs entered by the user, while they are trying to complete the minigame, and compare with the list containing the right sequence. If the sequence is correct when its size 4, so user complete the minigame and something happen in the gameplay, it depends on extension of this method in the development. If the sequence entered is wrong, the minigame quit without any change.

FINGERPRINT CLASS METHOD

It is the method used to start the minigame when the player press E with the required item.

```

public procedure start()

    system.background = fingersolution.png

    started = True

endprocedure

```

It just change the background to the background with the solution. The started control the state of the minigame, so the user can repeat multiple times to change the sequence. It will have also a extension of check().

HACKING CLASS METHOD

It will start the hacking minigame when the key E is pressed when the chip is in the player inventory.

```

public procedure start()

    system.background = hackingsolution.png

    started = True

endprocedure

```

It change the background to the background with a hint of the solution. In development section it will have an extension of check().

LOCKPICK CLASS METHOD

It will start the lockpick minigame when the E is pressed while the lockpick is in inventory.

```
public procedure start()

    for element in correct_sequence

        print element

        wait few time

        print screen.background

    next element

endprocedure
```

Temporally print the solution sequence and user should input it in same order. It will extend the check().

GUARD CLASS METHOD

It will have on method that will be used to find the next moves of the guards automatically, so it will run concurrently.

```
public procedure next_move()

    Dijkstra algorithm using the screen size and the wall and sprites position

    if LENGTH(nextMoves) < 5 then

        add the next move found by algorithm to nextMoves queue

    endif

endprocedure
```

The Dijkstra algorithm will find the shortest path to player to calculate the better next move; it will constantly change given that player is moving. I chose this algorithm because it's widely used in networking given its efficiency to find paths, furthermore it's covered in A level making easy to connect with our school content practice. The queue nextMoves have a limit of 5 because as the player move we need to find shortest paths to them, so few next moves are predicted to prevent delay.

LOCAL DATABASE

It's a global database used to store the username and user preferences; it's stored locally to decrease the number of server requests decreasing the server processing that could slow down the server.

```
CREATE TABLE user {

    username TEXT,

    upButton TEXT,
```

```
        downButton TEXT,

        leftButton TEXT,

        rightButton TEXT,

        useButton TEXT,

        interactButton TEXT

};
```

It will use only one table for all because it's just accessed once per session, so no needed of normalization to decrease the processing. One table make simple to introduce without worry about rules.

API CODE

CODE

The API server will have few code, only the necessary to get the information and check the database, so I won't use diagram or separate the code in parts. I will use Flask for this part because it's a simple tool that is implemented in Python, so no external tool needed. Flask allows the communication using POST methods by using the request Python library.

```
route the request of new user

function new_user()

    get the message

    decrypt the message

    check if username exist

    if username exist then

        return error

    else

        add username to database

        return success

    endif

endfunction
```

It is used to add the new username to the database if it doesn't exist, this request happens when we are in the account screen, and we confirm the username that we want. The encryption of data transfer should happen because man-in-the-middle attacks can happen, they could add malicious code to the legitimate packet sent by

user and the server. The code should return something to tell the distribution code if it was successful or failed. A server-side authentication should also happen to check possible SQL injection or XSS attack.

```
route the request of new highest score

function set_score()

    get the message

    decrypt it

    update the database with the score

    if error then

        return error

    endif

    return success

endfunction
```

It's used to set the best score of the user when it's sent to server. In this way all users in the world could see it if part of the 10 best scores in world.

```
route the request of the best 10 scores

function best_scores()

    find the best 10 scores from database

    return the best 10 scores as a list

endfunction
```

It's used to get the 10 best scores and return it to the distribution code, with this the ranking can be displayed. In all the functions above, the Python sqlite3 library is used to execute the commands in the database, and all data returned is encrypted before send to prevent man-in-the-middle attacks.

SERVER DATABASE

This database will have the data from diverse users worldwide. Given that multiple operations happen on it, we should normalize it in maximum as possible.

```
CREATE TABLE users {

    username TEXT PRIMARY KEY

};
```

Store all usernames used by people around the world, it's checked every time that a new username wants to be created. The username is a primary key because it's unique. Stored in different table case we need more user information in future, if it happens then we just increase the table.

```
CREATE TABLE maps {  
  
    ID INTEGER PRIMARY KEY,  
  
    name TEXT  
  
};
```

It stores the maps used in the game. There is a ID to identify them, it's used in normalization to do not store the same map name again and again.

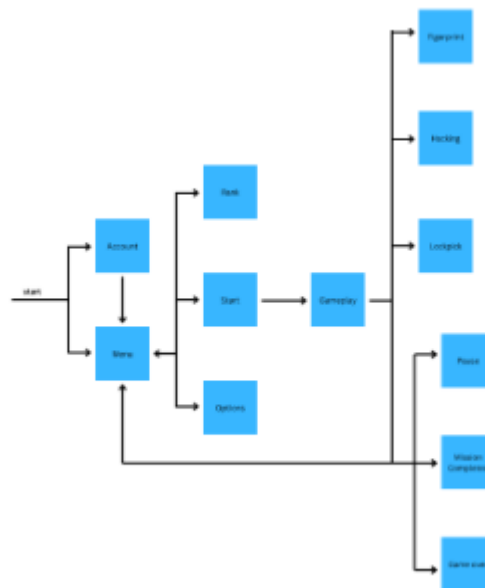
```
CREATE TABLE results {  
  
    username TEXT,  
  
    mapID INTEGER,  
  
    lives INTEGER,  
  
    timeTaken TEXT,  
  
    FOREIGN KEY username REFERENCES users (username),  
  
    FOREIGN KEY mapID REFERENCES maps (ID)  
  
};
```

It stores the number of lives and the time taken by the user in each map, the main definition of best score is the number of lives, as higher as possible, after is the time taken, as lower as possible. The tables are separated because it improve the operations time by using normalization.

GRAPHIC DESIGN

SCREENS

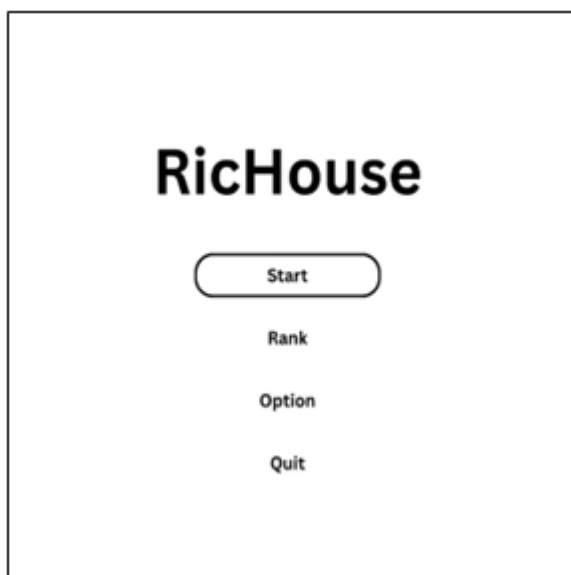
The screens will constantly change and each of them have its own elements to allow the user to comprehend what is happening in the code. The diagram shows how the screen are linked together. As said before, the screens will be squared, 600x600, because it makes simpler to implement Dijkstra algorithm during the gameplay.



The relationship is show by arrows, the point shows the next screen, so:

- Account goes to menu
- Menu go to Rank, Start or Options
- Rank, Start or Options go to Menu
- Start go to Gameplay
- Gameplay go to Hacking, Fingerprint, Lockpick, Pause, Mission completed or Game over
- Lockpick, Hacking, Fingerprint and pause go to Gameplay
- Pause, Mission completed and Game over go to Menu

MENU SCREEN



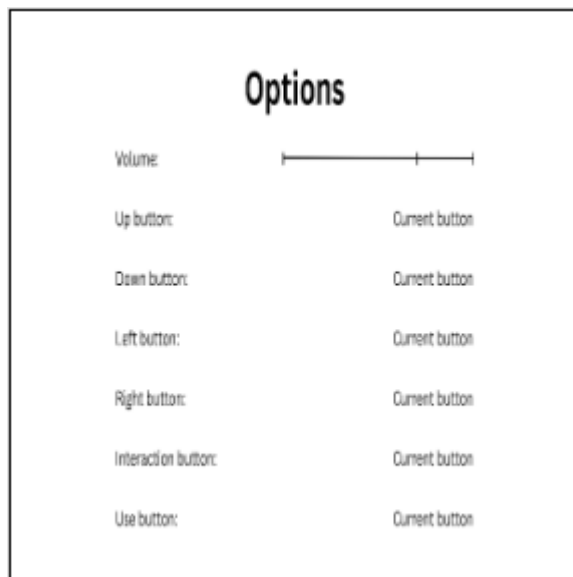
Buttons are centralized because it fixes the user attention to the middle of the screen while the side spaces can be fill by a beautiful art, in this way user can feel more immersion. The game title is the bigger text because in this way user can have sure that it's the title furthermore it's more important to identify the game than the buttons. Buttons are in vertical; it was a decision for all screens to allow me to reuse the general selection code in all screens with buttons. The box around the selected button is used to show the user what they are currently selecting, connection between computer logic and user understand.

RANKING SCREEN



There are 2 rankings because there are 2 different maps that take different amount of time, it's unfair to compare the time using only 1 rank. The top 10 was chose because it's more attractive than top 50 for example, the exclusivity can make the users play and effort more to participate in the ranking. It will look like the.

OPTIONS



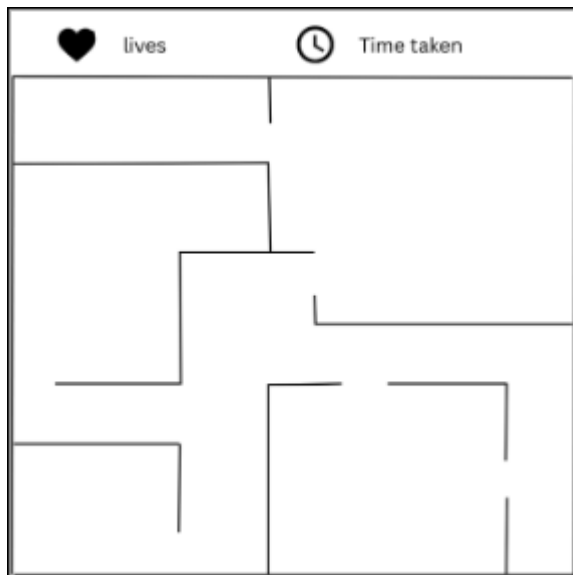
The screen used to change the buttons and volume preferences. The volume is shown by a bar because in this way we can visualize the audio intensity. The other options are the button preferences; it show what button it refers and in the right is the current button registered. The screen name is always print on top, it support the user identify where they entered.

START SCREEN



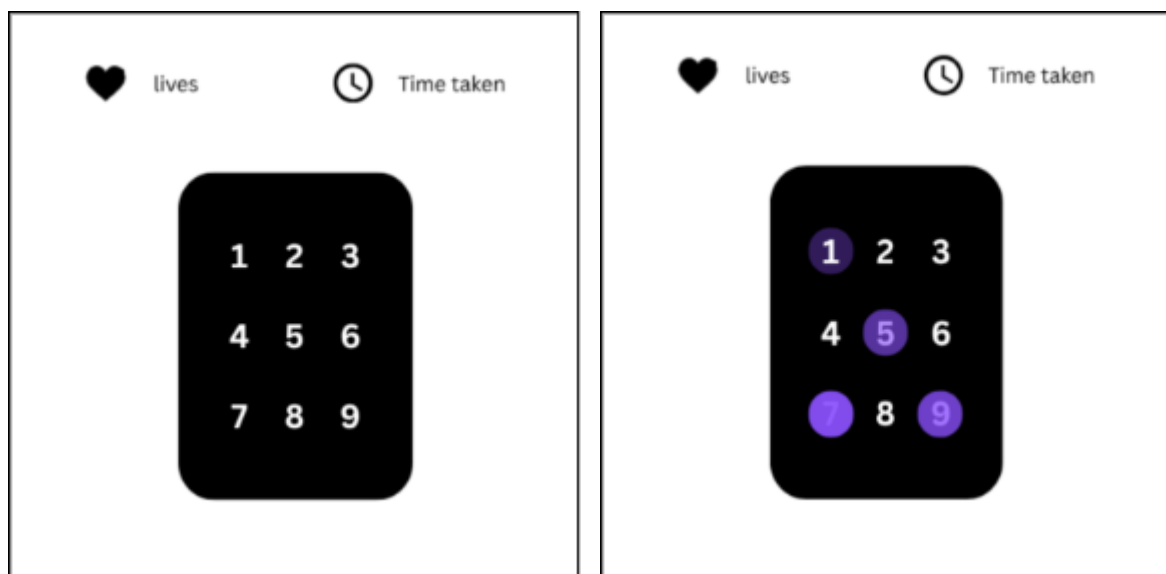
The screen used to choose the map before starting the gameplay. It will have 2 maps, and the background will be divided in 2, one showing a part of each map, in this way the user can see and choose the map. Again, the button selected has a box and the title showing the screen.

GAMEPLAY SCREEN



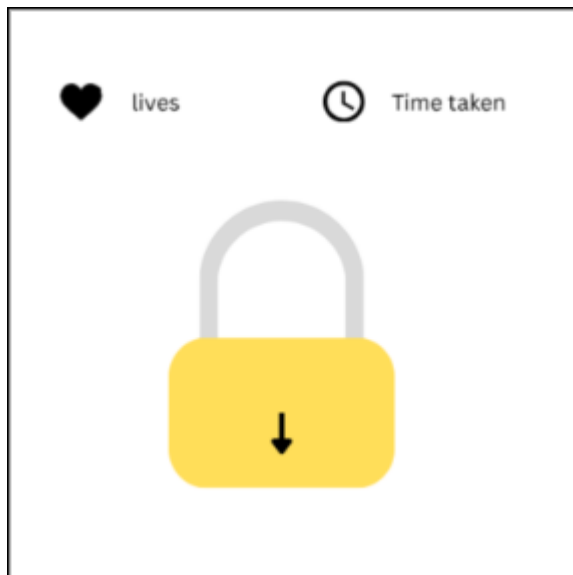
It's the structure of the gameplay screen; the maps will be different. The screen has the statuses on top, it shows the number of lives and the time taken, it supports the user to see its performance in the mission. The map has lines showing the walls, it connects the logic of walls in code with users' vision. The map is square to support the implementation of Dijkstra algorithm.

FINGERPRINT SCREEN



The first screen is when you enter in the minigame, it gives a background to user understand what's going on. The statuses still being printed because the guards will continue to look for player and the time still running. The second is when you start it with the necessary tool. The screen shows the sequence by increasing order of intensity, it looks like a fingerprint.

LOCKPICK SCREEN



It's the screen opened when the lockpick game is entered. In beginning the lock doesn't have anything inside, after starts it show a sequence of 4 arrow. The user should input the correct arrow sequence. The statuses continue to be printed because the gameplay is happening.

HACKING SCREEN



The screen when you enter in the hacking minigame. The screen when user enter in the minigame there is no word in it, after starting a word with messing characters is printed. User should input the missing characters in order. The statuses continue to be printed.

PAUSE SCREEN



When you press ESC during the gameplay. There are 2 options, the return to continue gameplay or menu to return to menu screen. Big title to know what's happening. The selected button is marked by a box.

GAME OVER SCREEN



The screen opened when you lost all the lives, and the mission fail. It's just a screen with a art as background and a big "Game over" to tell user what happened.

MISSION COMPLETED SCREEN



Appear when the player arrives in the front door with the golden statue. It will have an art showing the glory of success and a big mission completed to give a feel of success for the user.

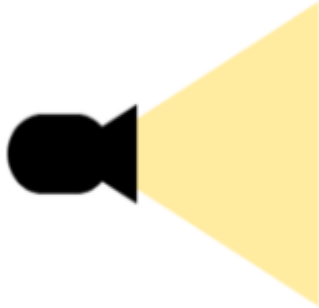
SPRITES

GUARDS AND PLAYER SPRITES



A simple character that will be used as base of player and guards' sprites. It has a cartoon and doesn't have complex details, it makes the user focus on gameplay instead of graphics, furthermore it's a good minimalist design. There's no mouth because the game doesn't have dialogues.

CAMERAS SPRITES



It's a simple sprite that will rotate during the game. The yellow part shows the range that camera can see the user, a way to connect the code logic and the user game view. It's simpler to combine with another map elements.

STAKEHOLDERS' FEEDBACK ABOUT THE LAYOUTS

- [Enzo Garofani:](#)

The GDD is good, it has an identity, the screens seem to make sense with this minimalist and well-executed gameplay noir. I hope the game follows a good identity like the one presented here.

- [Zach Ward:](#)

Waiting answer...

- [Alex Michalski:](#)

Waiting answer...

TESTING

WHITE BOX TESTING

It's the testing of the code structure, so it will optimize the code and its functionality. Useless lines can slow down the code, and poorly written conditions and Boolean expression can bug the game in some cases.

For this test I can check each module manually by getting each function and putting it in the main() to run individually. When it runs, I can check the lines being executed using debuggers that shows the current line being executed. This test won't test any possible input, only the structure.

I can also test using an automatic tool. In this case I would use the PyTest, it's a library that can be installed to check your code. It will automatize the process of white box testing using less time, in this way I could focus on another tests.

BLACK BOX TESTING

It's the testing for possible inputs entered by the user during the play. It checks the basic function ability of the game before distributing it in the Alpha and Beta testing. The inputs tested in each screen will be listed below.

Data	State	Expected result
Menu		
W	Valid	Up button selected, return to last button if top arrived
S	Valid	Down button selected, return to first button if bottom arrived
ENTER	Valid	Confirm the selected button, open the screen
Any other key	Invalid	The key doesn't do anything
Rank		
ESC	Valid	Return to the menu screen
Any other key	Invalid	The key doesn't do anything
Options		
W	Valid	Up button selected, return to last button if top arrived
S	Valid	Down button selected, return to first button if bottom arrived

A	Valid	It's valid if volume button selected, decrease volume by 5 if higher than 5, else doesn't do anything
D	Valid	It's valid if volume button selected, increase volume by 5 if lower than 100, else doesn't do anything
SHIFT + Letters A - Z	Valid	It's valid if the button selected is used to update preferences, it will be the button for the action chose
SHIFT + Numbers 0 - 9	Valid	It's valid if the button selected is used to update preferences, it will be the button for the action chose
ESC	Valid	Return to the menu screen
Any other key	Invalid	The key doesn't do anything
Start		
W	Valid	Up button selected, return to last button if top arrived
S	Valid	Down button selected, return to first button if bottom arrived
ENTER	Valid	Confirm the selected button, open the screen
ESC	Valid	Return to the menu screen
Any other key	Invalid	The key doesn't do anything
Fingerprint		
E pressed once	Valid	Start the minigame by changing the background screen
E pressed more than once	Invalid	The key E pressed more than once should do anything more
Input of 4 numbers 0-9	Valid	The result will be checked by comparing with correct sequence
Input of more than 4 numbers 0-9	Invalid	The extra keys inputted will be ignored.

Correct sequence of numbers	Valid	The minigame is completed and return to gameplay changing some sprite state
Incorrect sequence of numbers	Valid	The minigame is completed and return to gameplay without change sprites state
ESC	Valid	Return to the gameplay screen without change any sprite state
Any other inputs	Invalid	The key doesn't do anything
Hacking		
E pressed once	Valid	Start the minigame by printing the word with missing characters
E pressed more than once	Invalid	The key E pressed more than once should do anything more
Input of 4 letters A-Z	Valid	The result will be checked by comparing with correct sequence
Input of more than 4 letters A-Z	Invalid	The extra keys inputted will be ignored.
Correct sequence of letters	Valid	The minigame is completed and return to gameplay changing some sprite state
Incorrect sequence of letters	Valid	The minigame is completed and return to gameplay without change sprites state
ESC	Valid	Return to the gameplay screen without change any sprite state
Any other inputs	Invalid	The key doesn't do anything
Lockpick		
E pressed once	Valid	Start the minigame by printing the arrows sequence
E pressed more than once	Invalid	The key E pressed more than once should do anything more

Input of 4 arrows	Valid	The result will be checked by comparing with correct sequence
Input of more than 4 arrows	Invalid	The extra keys inputted will be ignored.
Correct sequence of arrows	Valid	The minigame is completed and return to gameplay changing some sprite state
Incorrect sequence of arrows	Valid	The minigame is completed and return to gameplay without change sprites state
ESC	Valid	Return to the gameplay screen without change any sprite state
Any other inputs	Invalid	The key doesn't do anything
Pause		
W	Valid	Up button selected, return to last button if top arrived
A	Valid	Down button selected, return to first button if bottom arrived
ENTER	Valid	Confirm the selected button and render the screen
Any other key	Invalid	The key doesn't do anything
Gameplay		
W	Valid	Move character five pixels up if there is no wall, else it is move to one pixel before the wall.
A	Valid	Move character five pixels left if there is no wall, else it is move to one pixel before the wall.
S	Valid	Move character five pixels down if there is no wall, else it is move to one pixel before the wall.
D	Valid	Move character five pixels right if there is no wall, else it is move to one pixel before the wall.

F	Valid	It will interact with minigame or searchable place if next to it, else do nothing
ESC	Valid	Open the pause screen
Open game over screen if lost all lives	Valid	Print the game over screen and return to menu
Open mission completed screen if arrive at door with statue	Valid	Print the mission completed screen and return to the menu
Lost a live if detected by security system	Valid	Lost one live and teleport user to front door
Any other input	Invalid	Do nothing
Account		
Enter a username smaller than 20 characters	Valid	The username is sent to API to check if it's available
Enter a username bigger than 20 characters	Invalid	The keys after 20 th character are ignored
Number 0-9	Valid	It enters in the username
Letters A-Z	Valid	It enters in the username
_ key	Valid	It enters in the username
Any other key	Invalid	It won't enter in the username

When I did this I consider security methods, when the input is checked in database or executed it can't have programming symbols to avoid SQL injections and XSS attacks.

ALPHA TESTING

In this stage I will play the game as a user looking for bugs and possible improvements in the gameplay.

First, I will try to open all screens to check if all screens load correctly. After I will try to play both maps, I will use the movement, interaction and use button to check the searchable places, enter and start in the minigames, and open the pause screen. I will check the guard's detection, and the mission completed and the game over screens.

After I will check if my results were stored and sent to API. To do it, I will check the rank because I will be the only player in the world, if it appears there it's working. Finally, I will try to change the preferences and play with the new preferences.

Check if in the account screen I can't use a username that has some coding character such as <, - and / that could be used in SQL injection or XSS attack

BETA TESTING

The stage in that stakeholders will have access to the game. They will play and give the feedback about the game, it's essential to find bugs and improve the features to make the game attractive. They should install the VS Code and the Pygame to be able to play the game, they don't need Flask because it is used in the server not in user machine. There are some files to be sent.

The first type of file will be the .py, it stores all Python code with containing the actual game logic. It will use Python version 3.12.0 to support features such as the match case and the `__main__` variable.

The other kind of file will be the .png, it will store the game screens and should be distributed with the code for Pygame be able to render it. Another file related with audiovisual effects is the .mp3 that will be files found on the Internet to use a music and effects without copyright.

The last type is a file .db, it's used to store the tables of the SQLITE. It's in binary, so if user can't use SQLITE they won't be able to access the tables. Even if they change the table, they won't be able to change their score on server, just locally.

PROJECT MANAGEMENT

I should divide the design section in parts for the development; it will turn the development steps easier and more organized.

To manage the files, I will use a VS Code environment in GitHub, it will allow me to access the code anywhere by accessing VS Code remotely. Another tool that will help me is the Git, it allows to save project versions, so I could access the past versions of the code with I think they're better than the current one.

Furthermore, put my code on web can benefit me to show the code for public, it can attract more users and support in feedback integration. I don't need to care about wide the code because it will be open source.

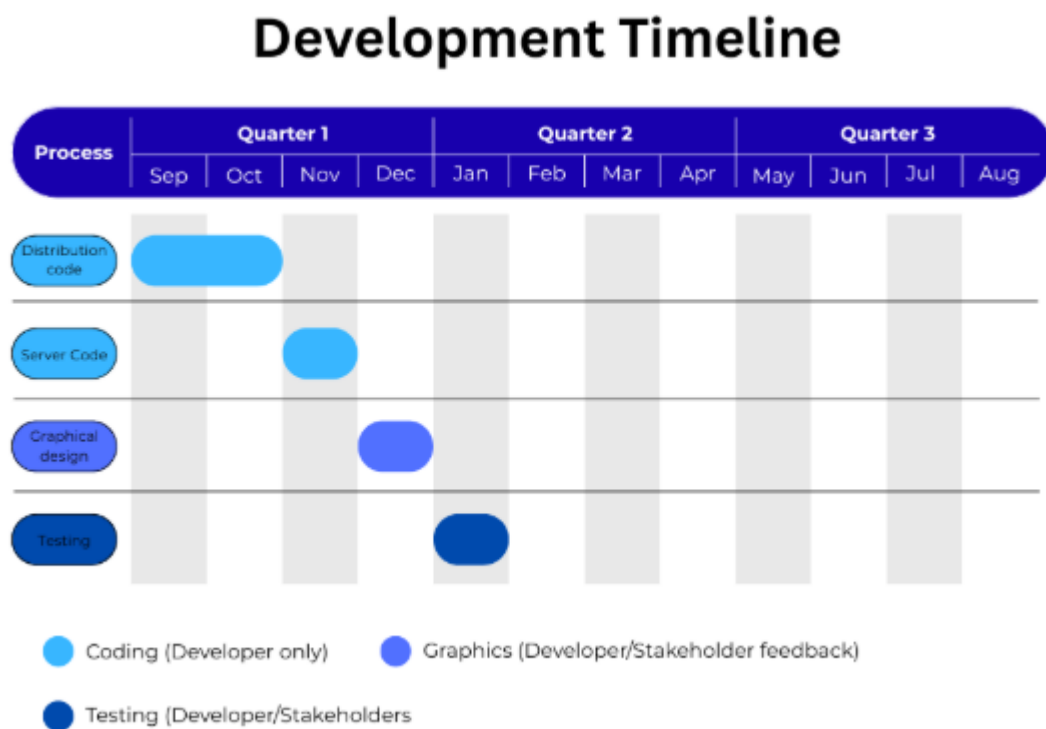
Here is the development plan:

Design part	Status
-------------	--------

Distribution code	
main() function	Not completed
game_loop() function	Not completed
input() function	Not completed
System class	Not completed
Button class	Not completed
Preferences class	Not completed
Gameplay class	Not completed
Minigame class	Not completed
Hacking class	Not completed
Fingerprint class	Not completed
Lockpick class	Not completed
Sprites class	Not completed
Player class	Not completed
Guards class	Not completed
Local database	Not completed
Server code	
new_user()	Not completed
set_score()	Not completed
best_score()	Not completed
Complete server database	Not completed
Put the code in some free Internet server	Not completed
Graphic design	
Account screen	Not completed
Menu screen	Not completed
Rank screen	Not completed
Option screen	Not completed
Start screen	Not completed
Map 1 gameplay screen	Not completed
Map 2 gameplay screen	Not completed
Lockpick screen	Not completed
Hacking screen	Not completed
Fingerprint screen	Not completed
Pause screen	Not completed

Mission completed screen	Not completed
Game over screen	Not completed
Testing	
White box testing	Not completed
Black box testing	Not completed
Alpha testing	Not completed
Beta testing	Not completed

I should thing in a way to divide efficiently the work, below I put a diagram showing the expected time for each part.



Now let's break out the weeks of each step. The diagram uses more weeks to handle possible lates in each part of development.,

Week 1	main(), game_loop() and input() functions
Week 2	System, Button and Preferences classes
Week 3	Gameplay and Minigame classes
Week 4	Hacking, Fingerprint and Lockpick classes
Week 5	Sprites and Player classes
Week 6	Guards class
Week 7	Local database
Week 8	Server database
Week 9	New_user() and set_score functions
Week 10	best_scores() function
Week 11	Add the code in a free server
Week 12	Account, Menu and Rank screens
Week 13	Options and Start screens
Week 14	Map 1 and Map 2 screens
Week 15	Lockpick, Hacking and Fingerprint screens
Week 16	Pause, Game over and Mission completed screens
Week 17	White box testing
Week 18	Black box testing
Week 19	Alpha testing
Weeks 20	Beta testing

There are some things that could late the project, during the coding I could use more time given that It's the first time using the Pygame tool. I can have problems to solve problems if I don't learn how to use the most important Pygame features. Another possible late is in stakeholders feedback, may be the stakeholders can use long times to give the feedback slowing down the process. Finally, in the resting the lates are more present given that I can find bugs and change features, it would take long times depending on the problem to be solved.

ETHICAL, LEGAL AND ENVIRONMENTAL CONSIDERATIONS

ETHICAL CONSIDERATIONS

The main consideration is about the accessibility, it's a fundamental part to make every user happy with my game, I can't just think in my style of gameplay.

The first thing to consider is the accessibility for the deaf persons. The game will have audios just to increase the immersion, so it won't affect on difficult or playability of the game. I can add more signal for specific actions, such as an alarm symbol in the middle of the screen when the alarm starts, it shows to use what happen before teleport the player. Unfortunately, I would not be able to adapt the game for blind persons, as I said the audio don't affect much in the play and the graphics are essential, in a stealth game the adaptivity for blind is a big issue.

Another accessibility option is the adaption of the game for special controllers, these controlled support persons with paralysis to play the game. I can adapt the option inputs, in this way special controllers or even normal controllers can be used, in this way I can expand the game for other devices too.

There is no considerations about age, the game won't have anything that could be prejudicial for children. It will be a family friendly game.

LEGAL CONSIDERATIONS

Let's start with cautions about the Data Protection Act. It's important to use encryption to transfer the users' data from server to user machine and vice versa, man-in-the-middle attacks can access their data or implement malicious code inside the files to access their computers. I should constantly update their data; an up-to-date data is essential for the accuracy of their information. I should handle just when it needed, I can't use their data for another purpose beside the purpose in that it was collected. When the user no longer wants to use my game, all it's data should be removed from server. Finally, I should use hashing if I use some password method, it improves the security inside the database.

Other legal consideration is the Copyright, Designs and Patents Act, it ensures that any intellectual property won't be copied. I should consider it when I use the .mp3 files that I will get in Google. It's crime to use any file with copyright in my game. I should see all terms and conditions of the resources used; some resources can't be used for commercial purpose.

ENVIRONMENTAL CONSIDERATIONS

The game will use a server, so I should be informed about the environmental effect of it even if the server isn't mine. In the server website, I should look for the environmental precautions, it will show the initiative of the company. Server uses loads of energy, so

look for places that uses solar panels is a good thing for example. It's essential to improve the company image in world.